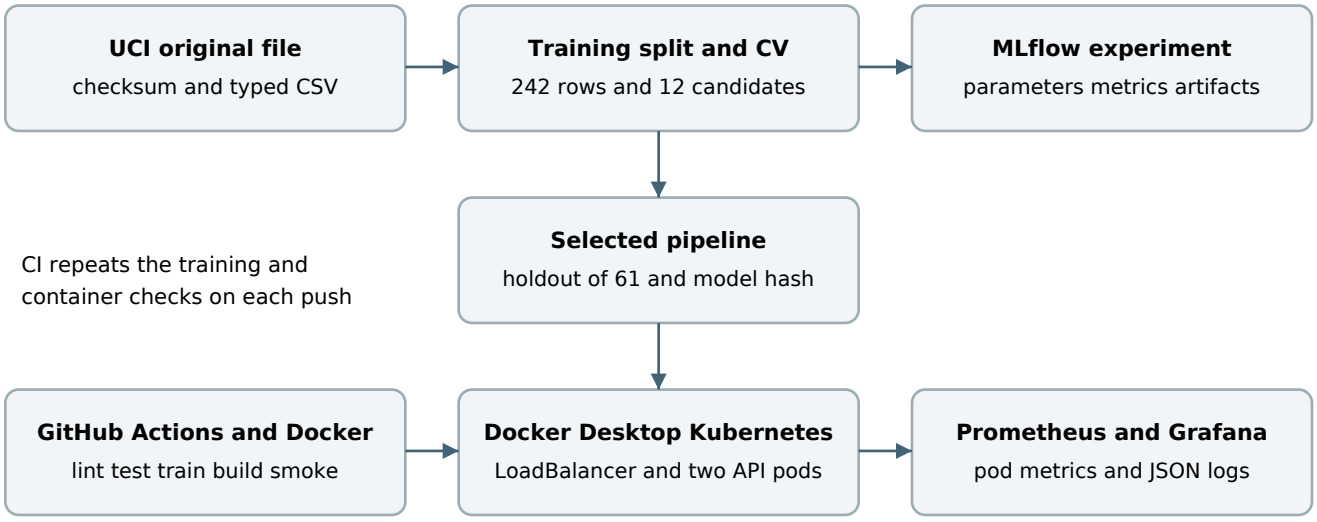
Heart Disease Classification MLOps Assignment 1

Machine Learning Operations AIMLCZG523
Prepared for Ameer Basha
11 October 2026

The project implements a reproducible classifier for the UCI Cleveland disease-status target. Data preparation, feature engineering, model comparison, experiment tracking, serving, container configuration and local Kubernetes monitoring form one connected pipeline.

Measured local result	Value
Dataset and split	303 records; 242 train and 61 test
Selected model	Logistic Regression; C 0.1; threshold 0.5
Holdout accuracy and ROC-AUC	0.9016 and 0.9643
Automated tests	29 passed



Architecture of the implemented project. Docker, hosted CI and Kubernetes execution require the laptop or repository run described in the setup guide.

Data, full training, MLflow, tests, the host API and its live monitoring dashboard were executed during preparation. The final audit tracks container, cluster and hosted CI evidence separately. The model output classifies the historical dataset; it is not a validated clinical risk score.

Repository: Add your actual GitHub URL after upload

Dataset acquisition and cleaning

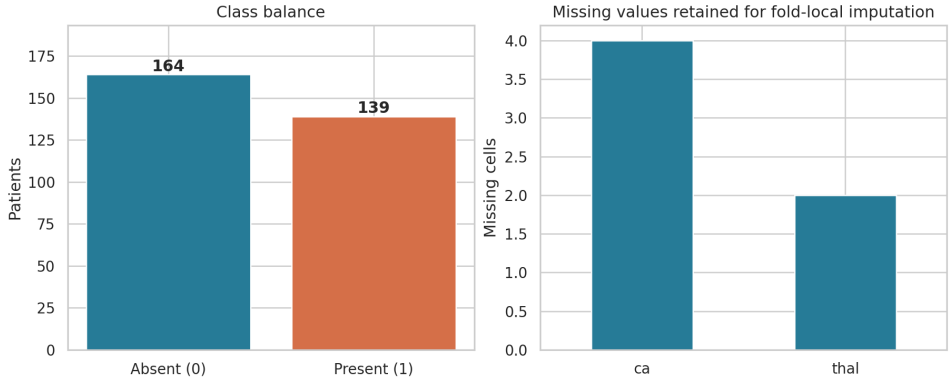
The source is the original processed Cleveland file from the UCI Heart Disease archive. The download script retrieves the archive, extracts the named file and verifies a fixed SHA-256. The bundled copy permits repeat runs without downloading the data again. The UCI attribution and CC BY 4.0 licence are retained in the repository [1].

Data contract	Observed value
Original and clean records	303 and 303
Target mapping	num 0 becomes 0; num 1 to 4 become 1
Class counts	164 absent and 139 present
Missing cells	ca 4; thal 2; all other fields 0
Duplicate rows removed	0

Predictor group	Fields and original codes
Numerical	age, trestbps, chol, thalach, oldpeak
Binary categories	sex, fbs, exang: 0 or 1
Chest pain and ECG	cp: 1 to 4; restecg: 0 to 2
ST slope and vessels	slope: 1 to 3; ca: 0 to 3
Thal test	thal: 3, 6 or 7

Cleaning converts question marks to missing values, validates categories, converts all fields to numeric types, maps the binary target and removes exact duplicate rows. No missing target is accepted. No training statistic is learned during this stage. The cleaned CSV deliberately retains nulls so each CV fold can fit its own imputer.

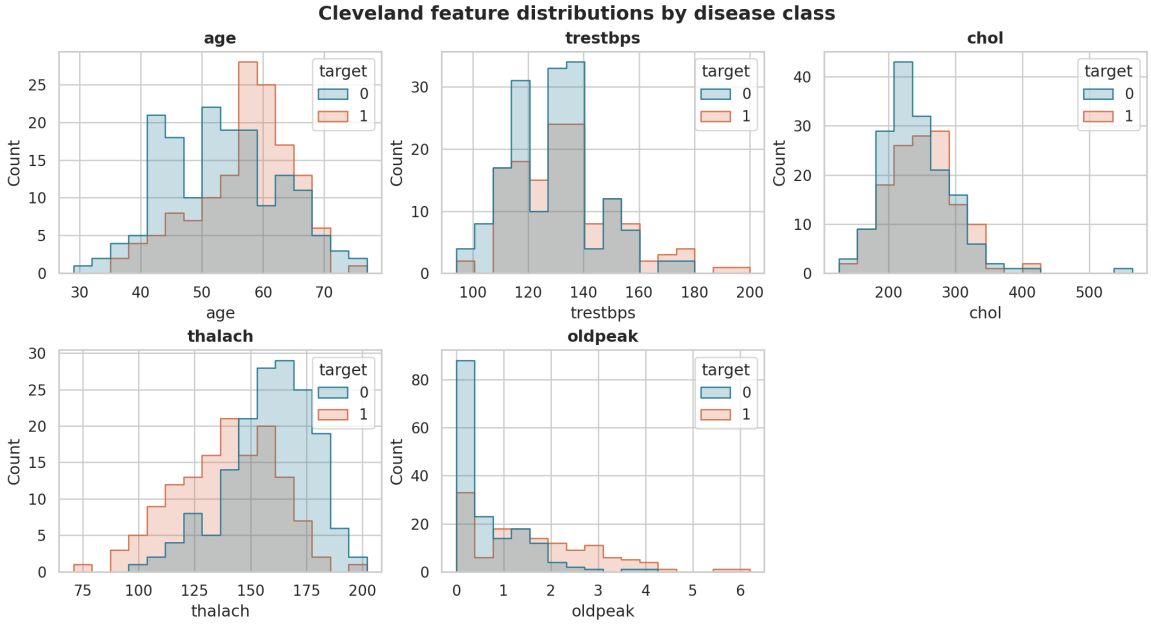
The source has 13 predictors and one diagnosis column. Two explicit, label-free interactions produce 15 modelling predictors before encoding. Additional patient measurements are not invented to meet a feature-count description.



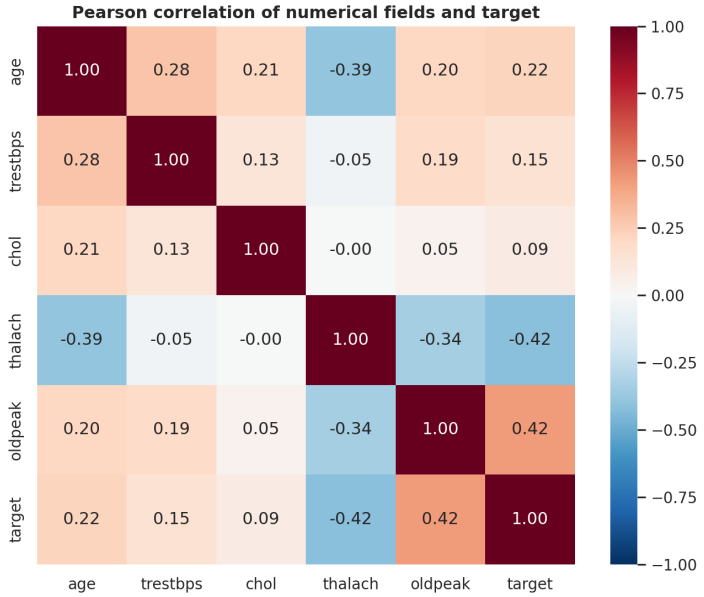
Observed class balance and missing-value counts from the included data.

Exploratory data analysis

The EDA script produces labelled distributions, class balance, missingness and a numerical correlation heatmap. It describes the complete dataset; supervised model selection subsequently uses only the training partition. Category codes are treated as categories in modelling rather than as continuous clinical measurements.



Histograms separate disease-status classes while retaining original measurement units.



Pearson associations among numerical fields and the binary target. These associations describe this cohort and do not establish causal effects.

The maximum achieved heart-rate field tends to associate negatively with the target, while oldpeak shows a positive association. The modest class imbalance supports stratified splitting and comparing class-weight choices. The small sample size and historical hospital cohort restrict the conclusions that can be transferred to new patients.

Features and model selection

FeatureBuilder adds age times oldpeak and exang times oldpeak without using labels. A ColumnTransformer median-imputes and standardises seven numerical fields, and mode-imputes and one-hot encodes eight categorical fields. The complete preprocessing and estimator are serialised as one pipeline, following the training-only fitting principle that prevents preprocessing leakage [2].

Logistic Regression compares C values 0.1, 1 and 10 with either no weighting or balanced weights. Random Forest uses 150 trees, balanced weights, depths 4, 8 or unlimited, and minimum leaf sizes 1 or 3. Both use seed 42. Each of the 12 candidates has five stratified training-only folds, with every preprocessing fit repeated inside the fold.

Model	Configuration	CV accuracy	CV AUC	AUC SD
LR	C=0.1; weight=none	0.8345	0.9015	0.0170
LR	C=1.0; weight=none	0.8430	0.9013	0.0163
LR	C=0.1; weight=balanced	0.8386	0.9011	0.0167
LR	C=1.0; weight=balanced	0.8470	0.8999	0.0162
RF	depth=4; leaf=3	0.8056	0.8970	0.0249
LR	C=10.0; weight=none	0.8388	0.8946	0.0262
LR	C=10.0; weight=balanced	0.8304	0.8946	0.0260
RF	depth=4; leaf=1	0.8139	0.8943	0.0242
RF	depth=8; leaf=1	0.8222	0.8937	0.0278
RF	depth=8; leaf=3	0.8180	0.8918	0.0280
RF	depth=none; leaf=1	0.8222	0.8913	0.0259
RF	depth=none; leaf=3	0.8180	0.8908	0.0237

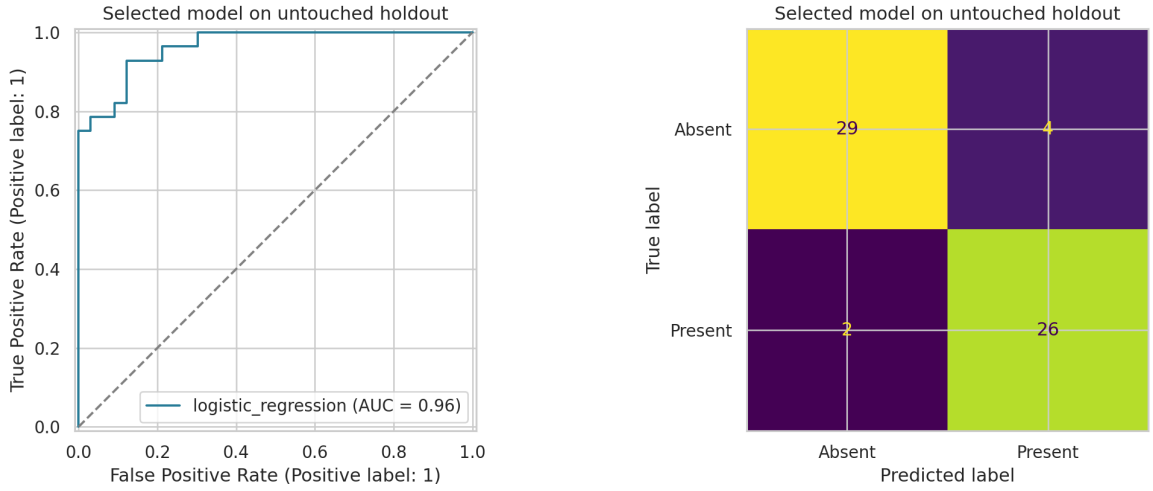
LR means Logistic Regression and RF means Random Forest. Values are fold means; AUC SD is the sample standard deviation across five folds. Accuracy, precision, recall, F1, ROC-AUC and Brier score are recorded for all candidates in model_comparison.csv.

Selection uses the highest mean CV ROC-AUC, not the held-out result or highest CV accuracy. The selected C=0.1 unweighted LR scored 0.9015 plus or minus 0.0170 AUC across folds. Differences among the leading settings are small relative to fold variation; this experiment does not establish a decisive general superiority.

Selected model evaluation

The selected pipeline is fitted on 242 training records and then evaluated once on the untouched 61-record holdout. The decision threshold remains 0.5. The held-out set is not used to choose hyperparameters or to tune the threshold. The same fitted pipeline is packaged for serving without a later all-data refit.

Held-out metric	Measured value
Accuracy	0.9016
Precision	0.8667
Recall	0.9286
F1	0.8966
Roc Auc	0.9643
Brier Score	0.0854



Holdout ROC and confusion matrix for the selected pipeline.

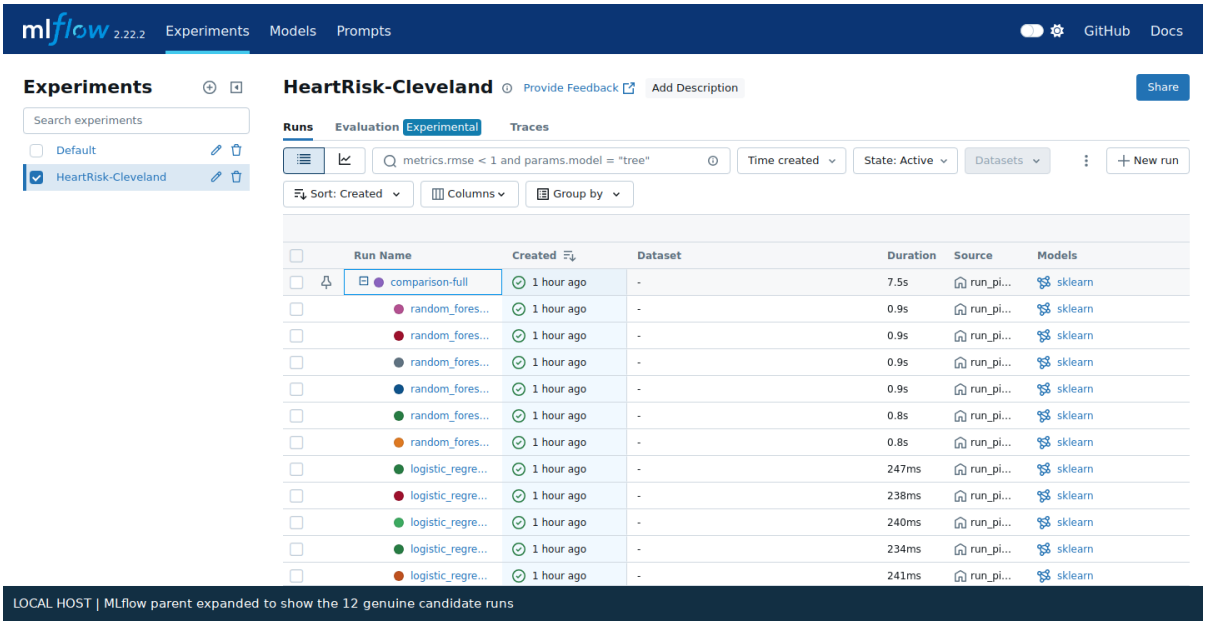
The confusion matrix contains 29 true negatives, four false positives, two false negatives and 26 true positives. This gives 55 correct predictions out of 61. Precision is 26/30 and recall is 26/28. AUC measures ranking over thresholds; the other classification metrics describe the fixed 0.5 decision threshold.

The API returns probability_disease and the probability of the chosen class as confidence. These probabilities have not been separately calibrated. Brier score is recorded as a probability-error measure, but one small holdout cannot establish reliable clinical calibration or generalisation. External and subgroup evaluation would be required before considering use beyond this educational dataset.

Experiment tracking with MLflow

A full execution creates one comparison parent run and 12 nested candidate runs. Each candidate records estimator parameters, per-fold metrics, metric means and standard deviations, its trained pipeline, CV details, an out-of-fold ROC plot and EDA figures. The parent stores the final holdout metrics, selected pipeline, data checksum, model metadata and comparison table.

Tracking field	Recorded evidence
Experiment	HeartRisk-Cleveland
Comparison run	55e0b783d27f45f594e45daaca954cbf
Selected candidate run	5612662ebdd244b88f611f4916dd0a91
Backend and artifact store	Local SQLite database and local artifact directory
Portable history	evidence/mlflow_runs.json and mlflow_runs.csv



Genuine browser capture of the executed local MLflow experiment.

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
          --host 127.0.0.1 --port 5000
```

Run the command on one line from the project root, then open localhost:5000. The training and UI processes must point to the same database. Machine-specific artifact paths are not shipped as a supposedly portable database; training on the final laptop recreates its live experiment and artifacts. The JSON/CSV export preserves the preparation run history independently of its absolute paths.

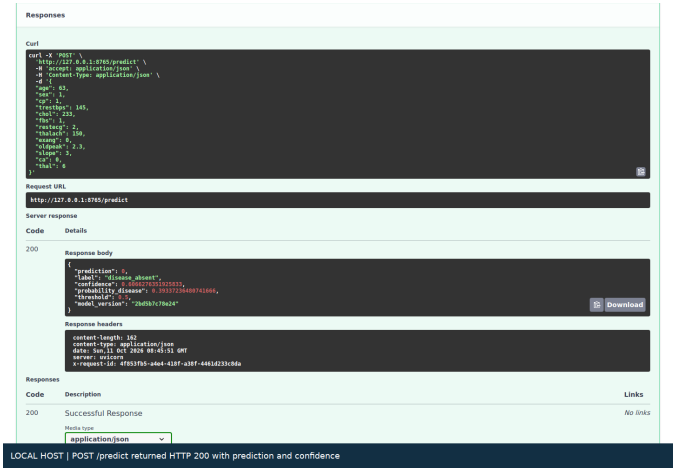
Packaging and API contract

The model package contains model.joblib and matching metadata.json. Startup verifies the saved SHA-256 before loading the model, and missing or inconsistent files fail readiness. Metadata records the input fields, threshold, split seed, dependency versions, selected run and results. The API loads the pipeline once and reuses it.

Endpoint	Contract
POST /predict	13-field JSON patient; class, confidence and model version
POST /predict/batch	1 to 100 patients; one response per patient
GET /health and /ready	Liveness and successfully loaded model readiness
GET /dashboard and /monitoring	Live process dashboard and JSON metric snapshot
GET /model-info and /metrics	Model metadata and Prometheus-format measurements


```
{
  "prediction": 0,
  "label": "disease_absent",
  "confidence": 0.6066276351925833,
  "probability_disease": 0.39337236480741666,
  "threshold": 0.5,
  "model_version": "2bd5b7c78e24"
}
```


Actual host response for examples/patient.json. Only ca and thal can be explicitly null; other fields are required and validated. Unknown fields, invalid categories, non-finite inputs and out-of-range values are rejected. The API accepts the original Cleveland codes, not the recoded categories used in some third-party CSV files.



Captured Swagger request. Swagger assets are served from the package.

requirements.txt includes training/test packages; requirements-serve.txt keeps the image smaller. Exact direct pins and constraints.txt record the validated dependency resolution. Host checks used Python 3.12.14; the Dockerfile uses the supported Python 3.11 family.

Automated testing and CI workflow

GitHub Actions runs on push, pull request or manual dispatch. Its sequential stages install dependencies, verify data, lint and check formatting, run unit tests, execute full training, validate the saved model, export MLflow records, build the image and black-box test its API. Unhandled failures stop the job; an always-run artifact step preserves available reports, logs, model files and tracking records.

Test area	Behaviour verified
Data	Checksum, categories, mapping, missing cells and invalid records
Model	Disjoint splits, fold-local imputation and serialisation parity
API	Schema errors, nulls, batch limits, metrics and model hash
Deployment contract	Replica count, ports, probes, LoadBalancer and pod scraping

Measured result: 29 tests passed. Unit coverage is 60% overall, including 97% of API code and 94% of feature code. The full EDA/training command is a separate integration execution; the unit coverage figure does not count all of that work. Actual tests.log, JUnit XML and coverage XML are included.

```
ruff check .                          All checks passed
ruff format --check .                 Formatting passed
pytest                                29 passed
python scripts/validate_model.py      Package and quality gates passed
```

The educational quality gates require holdout AUC at least 0.75 and accuracy at least 0.70. A hosted GitHub run additionally proves the isolated container stage. The workflow prepares a tested artifact; deployment to the laptop cluster is performed explicitly with deploy.ps1 because a hosted runner cannot directly reach the user's Docker Desktop context.

GitHub workflow: Hosted workflow execution evidence pending

Required final evidence: attach the successful Actions run screenshot from the actual repository. Local pytest output is not hosted CI proof.

Docker Desktop Kubernetes deployment

The Dockerfile builds a Linux serving image from Python slim, installs serving dependencies, includes the model and application, and runs as user 10001. A single Uvicorn worker per container keeps process metrics straightforward. The isolated smoke script checks the image on port 8001 without mounting the host model directory.

Deployment setting	Configuration
Cluster and namespace	Docker Desktop context docker-desktop; namespace heartml
Image and replicas	heart-risk-api:1.0; two API replicas
Service exposure	LoadBalancer port 8000 to named container port http
Health and resources	Startup/readiness /ready; liveness /health; 512 MiB limit
Pod security	Non-root; read-only root filesystem; dropped capabilities

```
.\scripts\docker_smoke.ps1
.\scripts\deploy.ps1 -SkipBuild
kubectl -n heartml get pods,svc,deploy
.\scripts\collect_evidence.ps1
```

The guide selects a single-node kubeadm cluster when offered. The local manifest uses imagePullPolicy Never, so the Docker Desktop runtime must see the built image. If local image visibility differs, the documented fallback publishes to the user's registry and changes the image name and pull policy together. The cluster remains Docker Desktop. API access is http://localhost:8000/docs [3, 4].

Container smoke evidence: pending laptop execution. Kubernetes endpoint evidence: pending laptop execution.

Capture two ready API pods, the LoadBalancer Service and an HTTP 200 prediction from that Service. The source preparation environment did not have Docker Desktop; no cluster execution is claimed here. Port forwarding can diagnose a pod, but does not replace LoadBalancer proof.

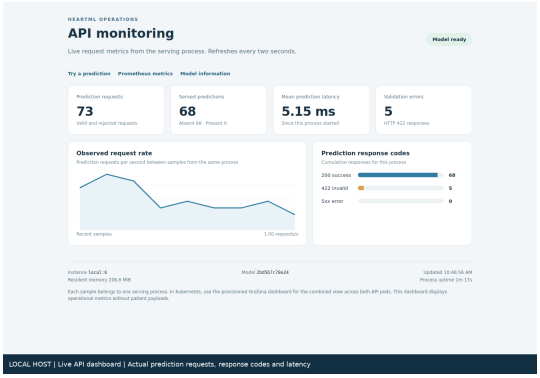
Monitoring and final evidence

Request middleware emits JSON route, status, duration and request-ID logs without patient payloads. Prometheus discovers each API pod separately using a namespace Role and ServiceAccount. Grafana provisions its datasource and six-panel dashboard from ConfigMaps. The traffic script sends valid samples and deliberate HTTP 422 requests to demonstrate the response-code panel [5, 6].

Measurement	Meaning
up	Number of available API pod scrape targets
HTTP request counter	Response codes and prediction request rate
Duration histogram	Prediction p95 over a two-minute window
Prediction counter	Served classes across replicas
Resident memory	Memory used by each serving process

Host-only API evidence: 30 sequential HTTP requests, mean latency 5.01 ms and observed p95 6.04 ms. This small local probe is not a container, cluster or capacity benchmark. Live host metrics are saved in evidence/metrics_sample.txt.

The built-in /dashboard displays actual request totals, class counts, response codes, mean latency and a request-rate chart. Its browser capture and JSON snapshot are included. It observes one serving process; Grafana aggregates the Kubernetes replicas.



Executed host API monitoring dashboard with real request traffic.

Run scripts/complete_assignment.ps1 on Docker Desktop to verify container, cluster and monitoring execution. Collect hosted CI, screenshots and the full video, then run scripts/finalize_submission.py --strict. The supplied video covers the host pipeline.

The lab uses temporary monitoring storage and local access. Production work would add persistent volumes, TLS/access control, immutable image releases, external model validation and labelled-outcome monitoring. Operational metrics alone do not detect classification accuracy or data drift.

References

[1] UCI Heart Disease dataset. Janosi et al. DOI 10.24432/C52P4X. archive.ics.uci.edu/dataset/45/heart+disease
[2] scikit-learn 1.5.2 common pitfalls and pipelines. scikit-learn.org/1.5/common_pitfalls.html
[3] Docker Desktop Kubernetes. docs.docker.com/desktop/use-desktop/kubernetes/
[4] Docker Desktop LoadBalancer exposure. docs.docker.com/compose/bridge/usage/
[5] Prometheus configuration. prometheus.io/docs/prometheus/latest/configuration/configuration/
[6] Grafana provisioning. grafana.com/docs/grafana/latest/administration/provisioning/